Student guide | Lesson 4 of 5

The growing tail

Each meal adds a tail segment that follows the head.

Name: _____ Date: _____

Today you will make

Every time the snake eats, a new tail segment appears and follows the head. Your game is starting to look like Snake.

Words you will meet

Word	What it means
list	A collection of things in order. segments = [] is an empty list.
append	Adds something to the end of a list.
for loop	Repeats code once for each item in a sequence.
range()	Makes a sequence of numbers to loop over.
len()	The number of items in a list.

The follow-the-leader idea

> Imagine a conga line of four people. The head steps forward. The person behind steps into the head's **old** spot, the next person into theirs, and so on. If the last person moves first, nobody lands on top of anyone else. That is why our loop starts at the tail and works forward.

Before you start

- Open snake3.py and save it as **snake4.py**.

Step 1: A list for the tail

Add this **after the food setup**:

```
segments = []
```

What it does

- This starts as an empty list. Each tail piece we create will be added to it.

Step 2: Add a segment

Add this **above game_loop**:

```
def add_segment():
    seg = turtle.Turtle("square")
    seg.color("green")
    seg.penup()
    segments.append(seg)
```

What it does

- It makes a new green square, just like the head.
- **segments.append(seg)** puts it at the end of the list.

Step 3: Move the body

Add this **above game_loop**:

```
def move_body():
    # Start at the tail so each segment copies the one in front of it
    for i in range(len(segments) - 1, 0, -1):
        segments[i].goto(segments[i - 1].pos())
    if segments:
        segments[0].goto(head.pos())
```

What it does

- Say the snake has three segments: A, B and C (C is the tail).
- C moves to where B is. Then B moves to where A is.
- **range(len(segments) - 1, 0, -1)** counts from the tail down to 1.
- Finally the first segment, A, moves to where the head is. **if segments:** stops an error when the list is empty.

Step 4: Grow when eating, and move the body

Update game_loop. The new lines are marked. **move_body()** must go **before** move_head().

```
def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        head.goto(0, 0)
        head.direction = "stop"

    if head.distance(food) < STEP:
        move_food()
        add_segment()          # new

    move_body()                # new
    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)
```

Try it: Run it. Eat some food. A green tail should grow and follow you.

Step 5: Stop the snake reversing into itself

Now that there is a tail, replace your four steering functions with these:

```
def go_up():
    if head.direction != "down": head.direction = "up"
def go_down():
    if head.direction != "up": head.direction = "down"
def go_left():
    if head.direction != "right": head.direction = "left"
def go_right():
    if head.direction != "left": head.direction = "right"
```

What it does

- Each one only turns if you are not going the opposite way.
- Pressing Down while moving up is now ignored.

Try it: Run it. Try to reverse. The snake should ignore you.

Heads-up: if your snake hits the wall now, the head jumps to the middle but the tail stays behind. That is expected. Lesson 5 fixes it.

Think about it: What would go wrong if move_body() ran **after** move_head()?

Your answer: _____

Challenges (pick one, or do them all)

- [ ] Make each new segment a different colour.
- [ ] Print len(segments) each time the snake eats. Where would you put the print?
- [ ] Give the head a different shape from the tail.

Stuck? Check here first

What you see	Likely cause and fix
The snake never grows	add_segment() is not called inside the distance if block.
The tail lags or jumps oddly	move_body() must come before move_head().
The tail is a clump of squares	Check range(len(segments) - 1, 0, -1).
Reverse key does nothing	That is the new Step 5 behaviour.

I can...

- [ ] store many objects in a list and add to it with append.
- [ ] explain why the tail moves from back to front.
- [ ] create new turtles while the program is running.
- [ ] use a for loop with range.

Check your work: the complete code

Compare your file with this one, line by line. If yours does not run, the mistake is usually a typo, a missing colon, or indentation that is not lined up.

```
import turtle
import random

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"

food = turtle.Turtle("circle")
food.color("red")
food.penup()
food.goto(0, 100)

segments = []

def go_up():
    if head.direction != "down": head.direction = "up"
def go_down():
    if head.direction != "up": head.direction = "down"
def go_left():
    if head.direction != "right": head.direction = "left"
def go_right():
    if head.direction != "left": head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def move_body():
    # Start at the tail so each segment copies the one in front of it
    for i in range(len(segments) - 1, 0, -1):
        segments[i].goto(segments[i - 1].pos())
    if segments:
        segments[0].goto(head.pos())

def add_segment():
    seg = turtle.Turtle("square")
    seg.color("green")
    seg.penup()
    segments.append(seg)

def move_food():
    food.goto(random.randrange(-LIMIT, LIMIT, STEP),
              random.randrange(-LIMIT, LIMIT, STEP))

def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        head.goto(0, 0)
        head.direction = "stop"
```

```
    if head.distance(food) < STEP:
        move_food()
        add_segment()

    move_body()
    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

game_loop()
screen.mainloop()
```